МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ПРОГРАММНОЙ ИНЖЕНЕРИИ

КУРСОВОЙ ПРОЕКТ
ЗАЩИЩЕН С ОЦЕНКОЙ

РУКОВОДИТЕЛЬ

доц., к.т.н		В. А. Матьяш
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОМУ ПРОЕКТУ

ИСПОЛЬЗОВАНИЕ ЗАДАННЫХ СТРУКТУР ДАННЫХ И АЛГОРИТМОВ ПРИ РАЗРАБОТКЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

по дисциплине: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР.	4236		А. Панченко
		подпись, дата	инициалы, фамилия

Санкт-Петербург 2024

Содержание

1. ЗАДАНИЕ И ЦЕЛЬ НА КУРСОВОЙ ПРОЕКТ 3
2. ВВЕДЕНИЕ 4
3. АЛГОРИТМЫ И СТРУКТУРЫ ДАННЫХ 5
3.1 СТРУКТУРА ДАННЫХ FLIGHTDATA 5
3.2 ДЕРЕВО ДЛЯ АВИАРЕЙСОВ 6
3.3 СТРУКТУРА ДАННЫХ ДЛЯ БИЛЕТОВ 7
3.4 СОРТИРОВКА РАСПРЕДЕЛЕНИЕМ 9
3.5 МЕТОД ОТКРЫТОГО ХЕШИРОВАНИЕ ХЕШ-ТАБЛИЦЫ 11
3.6 МЕТОД ОБРАТНОГО ОБХОДА ДЕРЕВА 13
3.7 БОУЕРА И МУРА(БМ) 15
4. ОПИСАНИЕ ПРОГРАММЫ 18
5. ТЕСТИРОВАНИЕ ПРОГРАММЫ 20
ЗАКЛЮЧЕНИЕ 31
СПИСОК ЛИТЕРАТУРЫ 32
ПРИЛОЖЕНИЕ 1 33

1. Задание и цель на курсовой проект

Цель курсового проектирования: изучение структур данных и алгоритмов их обработки, а также получение практических навыков их использования при разработке программ.

Разработать программу “Продажу авиабилетов” с дополнительными функциями:

- Последние три цифры ст. билета: 832
- Предметная область: 4) Продажа авиабилетов
- Метод хеширования: 0) Открытое хеширование
- Метод сортировки: 6) Распределением
- Вид списка: 2) Циклический однонаправленный
- Метод обхода дерева: 1) Обратный
- Алгоритм поиска слова в тексте: 0) Боуера и Мура (БМ)

2. Введение

Задача продажи авиабилетов в современном мире является одной из ключевых в индустрии путешествий. Для эффективной работы с данными о билетах и клиентах необходимо использовать оптимальные методы хеширования, сортировки, хранения списков и обхода деревьев. В данном контексте применение открытого хеширования, метода сортировки распределением, циклического однонаправленного списка, обратного метода обхода дерева и алгоритма поиска слова в тексте по Боуэру и Муру (БМ) является необходимым для обеспечения быстрой и эффективной работы системы продажи авиабилетов. Комбинация указанных методов позволит оптимизировать процессы хранения, обработки данных и повысить качество обслуживания клиентов.

3. Алгоритмы и структуры данных

3.1) Структура данных FlightData:

Рейс – это событие, связанное с перемещением транспортного средства из одного пункта в другой. Для эффективного хранения и управления информацией о рейсах может использоваться структура данных, содержащая следующие поля:

1. Идентификатор рейса: Уникальный номер или код, который служит для однозначной идентификации каждого рейса.
2. Точка отправления: Место, откуда начинается рейс, часто представленное названием аэропорта, станции или другими географическими координатами.
3. Точка назначения: Место, куда направлен рейс, также представленное названием аэропорта, станции или координатами.
4. Дата отправления: Календарная дата, когда рейс запланирован для отправления.
5. Время отправления: Часы и минуты, когда рейс должен начаться.
6. Общее количество мест: Общее число доступных мест для пассажиров или груза на транспортном средстве.
7. Доступные места: Текущее количество свободных мест, которые ещё не заняты пассажирами или грузом.

Структура данных FlightData представляет собой информацию о рейсе, включающую следующие поля:

- flightNumber (номер рейса): строковое значение для идентификации конкретного рейса.

- airlineName (название авиакомпании): строковое значение, указывающее на название авиакомпании, осуществляющей данный рейс.

- departureAirport (аэропорт вылета): строковое значение, обозначающее аэропорт отправления рейса.

- arrivalAirport (аэропорт прибытия): строковое значение, указывающее на аэропорт назначения рейса.

- departureDate (дата вылета): строковое значение, определяющее дату отправления рейса.

- departureTime (время вылета): строковое значение, указывающее на время вылета рейса.

- totalSeats (общее количество мест): целочисленное значение, отражающее общее количество доступных мест на борту самолета.

- availableSeats (доступные места): целочисленное значение, показывающее количество свободных мест на данный момент.

Структура данных FlightData предназначена для хранения информации о конкретном рейсе, что обеспечивает удобный доступ к необходимым данным для управления бронированием авиабилетов и отслеживания доступных мест на рейсе.

3.2) Дерево для авиарейсов

Дерево AVL представляет собой тип бинарного дерева поиска, в котором для каждой вершины высоты её двух поддеревьев различаются не более чем на единицу. Это сбалансированное дерево, где балансировка достигается автоматически путём вращений вокруг узлов.

Деревья AVL являются важной структурой данных, используемой в реализации многих операций баз данных, поиска и сортировки, где важна эффективность операций и минимизация времени доступа к данным.

Для реализации структуры дерева поиска в курсовом проекте была создана структура данных TreeNode, которая содержит информацию о рейсе (FlightData), указатели на левого и правого потомков.

Алгоритмы, используемые в курсовом проекте для работы с деревом поиска, включают следующие:

1. Вставка элемента в дерево:

Алгоритм состоит из следующих шагов:

- Начинаем с корневого узла

- Пока не найден лист для вставки нового элемента:

- Если новый элемент меньше текущего узла, переходим к левому потомку, иначе к правому

- Вставляем новый элемент в найденное место

2. Поиск элемента в дереве:

Алгоритм поиска осуществляет обход дерева с учетом значения искомого элемента. Если элемент найден, возвращается указатель на узел с этим элементом, иначе возвращается nullptr.

3. Удаление элемента из дерева:

Алгоритм удаления элемента включает следующие шаги:

- Если узел с удаляемым элементом является листом, просто удаляем его

- Если у узла есть только один потомок, заменяем узел его потомком

- Если у узла есть два потомка, находим наименьший элемент в правом поддереве, заменяем текущий элемент найденным наименьшим и удаляем его из правого поддерева

3.3) Структура данных для билетов

Билет – это документ, подтверждающий право пассажира на посадку на транспортное средство, такое как самолет, поезд или автобус. Для эффективного хранения и управления информацией о билетах может использоваться структура данных, содержащая следующие поля:

1. Номер паспорта пассажира: Уникальный идентификатор пассажира, представленный номером паспорта. Этот идентификатор позволяет связать билет с конкретным пассажиром.

2. Номер рейса: Уникальный идентификатор рейса, на который приобретен билет. Это поле ассоциирует билет с конкретным транспортным средством и маршрутом.
3. Номер билета: Уникальный номер билета, который служит для однозначной идентификации в системе учета и бронирования.

Эта структура данных может быть использована в системах бронирования и учета билетов для обеспечения эффективного отслеживания информации о пассажирах, рейсах и самих билетах. Важно отметить, что она может быть расширена дополнительными полями в зависимости от конкретных потребностей системы.

Ticket — это структура данных, предназначенная для хранения информации о билете. Включает следующие поля:

1. Номер паспорта пассажира (passportNumber): Строковая переменная, содержащая уникальный идентификатор пассажира в виде номера паспорта. Этот идентификатор позволяет связать билет с конкретным пассажиром.
2. Номер рейса (flightNumber): Строковая переменная, предназначенная для хранения номера рейса, на который приобретен билет. Это поле позволяет ассоциировать билет с конкретным авиарейсом.
3. Номер билета (ticketNumber): Строковая переменная, представляющая уникальный идентификатор билета. Каждый билет имеет свой уникальный номер для идентификации в системе бронирования или учета.

```
struct Ticket {
    string passportNumber;
    string flightNumber;
```

```
    string ticketNumber;
};
```

Эта структура данных предназначена для использования в системах учета и обработки билетов, обеспечивая связь между пассажирами и конкретными авиарейсами. Важно отметить, что эта структура может быть дополнена другими полями в зависимости от конкретных требований системы и необходимости хранения дополнительной информации о билетах.

3.4) Сортировка распределением

Алгоритм сортировки методом распределения заключается в разбиении исходного массива на несколько подмассивов, каждый из которых сортируется отдельно, а затем объединяется в результирующий отсортированный массив.

Основные шаги алгоритма сортировки методом распределения:

1.Определить, на какое количество подмассивов будет разбиваться исходный массив. Это можно сделать на основе значений элементов исходного массива или фиксированным числом.

2.Разбить исходный массив на указанное количество подмассивов.

3.Отсортировать каждый подмассив, используя другой метод сортировки, например, сортировку вставками или быструю сортировку.

4.Объединить отсортированные подмассивы в один отсортированный массив.

Преимущества алгоритма методом распределения включают в себя эффективную работу на больших объемах данных, возможность распараллеливания алгоритма и потенциально лучшую производительность на некоторых типах данных, например, когда имеются множественные одинаковые ключи. Однако неэффективен при малых объемах данных и может потреблять больше памяти из-за создания дополнительных подмассивов.

Для сортировки списка билетов по номеру билета был использован алгоритм сортировки методом распределения - bucket sort.

Bucket sort разбивает входной массив на отдельные 'ведра' или 'корзины' (buckets), каждая из которых хранит элементы с определенным диапазоном значений. В данном случае мы сортируем список билетов по номеру билета, то есть каждое 'ведро' содержит билеты с одинаковым номером.

```
// Функция для сортировки списка методом распределения
void bucketSort(vector<Ticket>& tickets) {
    sort(tickets.begin(), tickets.end(),
        [](const Ticket& a, const Ticket& b) {
            return a.ticketNumber < b.ticketNumber;
        });
}
```

Алгоритм bucket sort для данной задачи можно описать следующим образом:

1. Создаем определенное количество пустых 'ведер' (buckets), например, равное диапазону значений номеров билетов.

2. Распределяем элементы списка по 'ведрам' в соответствии с их номером билета.

3. Сортируем элементы в каждом 'ведре', например, с помощью стандартной функции сортировки std::sort.

4. Объединяем отсортированные 'ведра' в один отсортированный список.

Функция bucketSort принимает ссылку на вектор билетов и сортирует его по номеру билета, используя лямбда-функцию в качестве критерия сортировки.

Реализация данного алгоритма в вычислительных машинах обеспечивает эффективную сортировку больших объемов данных по номеру билета, обеспечивая быструю обработку информации о билетах.

3.5) Метод открытого хеширование (метод цепочек) хеш-таблицы

Метод открытого хеширования с использованием цепочек - это один из подходов к реализации хеш-таблиц, предназначенный для разрешения коллизий. Коллизия возникает, когда два или более ключа хешируются в одно и то же значение хеш-функции. Метод цепочек предлагает решение этой проблемы, разрешая коллизии путем создания "цепочек" элементов данных в одной и той же ячейке хеш-таблицы.

Вот теоретическое описание метода открытого хеширования с цепочками:

1. Хеш-таблица: Это структура данных, содержащая фиксированное количество слотов (ячеек), каждый из которых может хранить пару ключ-значение.
2. Хеш-функция: Это функция, которая принимает ключ и возвращает индекс в хеш-таблице, где должно быть сохранено значение. Хорошая хеш-функция стремится распределять ключи равномерно по всему диапазону индексов.
3. Цепочки: Каждая ячейка хеш-таблицы содержит не просто одно значение, а цепочку значений (список, массив и т. д.). Если два ключа хешируются в одно и то же значение, их значения добавляются в цепочку для этого индекса.
4. Вставка: При вставке нового элемента с ключом и значением:
   - Применяется хеш-функция для определения индекса в таблице.
   - Значение добавляется в конец цепочки в соответствующем индексе.
5. Поиск: При поиске элемента с заданным ключом:
   - Применяется хеш-функция для определения индекса в таблице.

- Производится поиск в цепочке по этому индексу.

6. Удаление: При удалении элемента с заданным ключом:
   - Применяется хеш-функция для определения индекса в таблице.
   - Производится поиск в цепочке по этому индексу, и элемент удаляется из цепочки.

Преимущества метода открытого хеширования с цепочками включают простоту реализации, эффективное использование памяти и способность обрабатывать коллизии без необходимости перемещения элементов в другие ячейки. Недостатками могут быть дополнительные затраты на хранение указателей или ссылок в цепочках, а также возможное замедление при поиске, если цепочки становятся длинными.

В курсовом проекте был использован метод открытого хеширования с цепочками для разрешения коллизий.

Структура данных KeyValuePair хранит пару ключ-значение (такие поля, как key, placeAndDate, fullName, birthDate и isDeleted).

Для хеширования ключей была использована функция Hash, которая принимает ключ типа charm1 и возвращает хеш-значение в виде целого числа.

Алгоритм метода открытого хеширования с цепочками можно описать следующим образом:

1. Используется хеш-функция для определения индекса в хеш-таблице, куда нужно поместить элемент.

2. Если по вычисленному индексу уже есть элемент, который занимает эту позицию, то создается связанный список (цепочка), где каждый элемент связан со следующим по указателю.

3. Новый элемент добавляется в конец цепочки, чтобы избежать коллизий.

4. По заданному ключу можно быстро найти нужное значение, просматривая элементы в цепочке, начинающейся с индекса, посчитанного хеш-функцией.

Реализация метода открытого хеширования с цепочками в вычислительных машинах обеспечивает эффективное разрешение коллизий и быстрый доступ к данным по ключу. Этот метод является гибким и хорошо справляется с большими объемами данных, позволяя хранить и быстро находить связанную информацию.

3.6) Метод обратного обхода дерева

Метод обратного обхода дерева представляет собой один из способов обхода бинарного дерева. При обратном обходе дерева посещение узлов происходит в следующем порядке: сначала левое поддерево, затем узел, и, наконец, правое поддерево. Этот порядок обеспечивает постфиксную запись выражений.

Принцип обратного обхода:

1. Посещение левого поддерева.
2. Посещение узла.
3. Посещение правого поддерева.

Таким образом, если применить обратный обход к бинарному дереву, результат будет упорядоченным списком узлов, следующим по возрастанию, если дерево является бинарным поисковым деревом.

```
void reverseInorderTraversal(TreeNode* node, std::vector<FlightData>& leftPart, std::vector<FlightData>& rightPart) {
    if (node == nullptr)
        return;

    // Обход левой части: начинайте с левого поддерева
    reverseInorderTraversal(node->left, leftPart, rightPart);
    // Если текущий узел меньше корня, добавляем в левую часть
    if (node->data.flightNumber < root->data.flightNumber) {
```

```
        leftPart.push_back(node->data);  // Сохраняем значение узла
    }
    // Если текущий узел больше корня, добавляем в правую часть
    else if (node->data.flightNumber > root->data.flightNumber) {
        rightPart.push_back(node->data);  // Сохраняем значение узла
    }

    // Обход правой части: начинайте с правого поддерева
    reverseInorderTraversal(node->right, leftPart, rightPart);
}
```

Обратный обход вершин (reverse inorder traversal) представляет собой способ обхода бинарного дерева, при котором сначала рекурсивно обрабатывается правое поддерево, затем текущая вершина, а затем левое поддерево.

В представленной функции reverseInorderTraversal происходит обратный обход вершин бинарного дерева. На каждом узле проверяется значение flightNumber текущего узла относительно значения flightNumber корня дерева (root). Если значение flightNumber текущего узла меньше значения flightNumber корня, то данные этого узла добавляются в левую часть (leftPart), иначе данные добавляются в правую часть (rightPart). Для обхода используются рекурсивные вызовы функции для левого и правого поддеревьев вершины.

Таким образом, данная функция позволяет собирать данные из дерева в соответствии с заданным порядком обхода, распределяя их между левой и правой частями в зависимости от их отношения к корню.

3.7) Боуера и Мура(БМ)

Метод Боуера-Мура — это эффективный алгоритм для поиска подстроки в строке. Он отличается от многих других алгоритмов поиска подстроки тем, что начинает поиск с конца подстроки, а не с её начала. Это позволяет существенно сократить количество сравнений, особенно в случае несоответствия символов.

Основные идеи метода Боуера-Мура:

1. Шаг сдвига: Алгоритм производит сравнение подстроки с текстом справа налево, начиная с конца подстроки. Если встречается несоответствие символа, то используется предварительно вычисленный сдвиг, который позволяет пропустить несколько символов в тексте. Это осуществляется на основе предварительно подсчитанных таблиц сдвига для каждого символа алфавита.
2. Таблица суффиксов: Для каждого символа алфавита в подстроке вычисляется максимальное смещение, на которое подстрока может быть сдвинута вправо, если в тексте несоответствие произошло на этом символе. Эта таблица позволяет эффективно выбирать оптимальные сдвиги в процессе поиска подстроки.
3. Префикс-суффиксное совпадение: При несоответствии символа в процессе сравнения, алгоритм анализирует подстроку, чтобы определить, есть ли в ней префикс, совпадающий с суффиксом текста. Если такое совпадение найдено, алгоритм осуществляет сдвиг так, чтобы совпавшие символы совпадали в следующем сравнении.

Преимущества метода Боуера-Мура включают его высокую скорость выполнения на практике и относительную простоту реализации. Этот метод широко применяется в поиске подстроки в тексте и часто используется в

различных программах, включая текстовые редакторы, поисковые системы и компиляторы.

```
int boyerMooreSearch(const string& text, const string& pattern) {
    int textLength = text.length();
    int patternLength = pattern.length();

    if (patternLength == 0) return 0;

    const int CHAR_SET_SIZE = 256; // Размер алфавита ASCII
    int lastOccurrence[CHAR_SET_SIZE]; // Таблица последних вхождений

    // Заполняем таблицу последних вхождений
    for (int i = 0; i < CHAR_SET_SIZE; ++i) {
        lastOccurrence[i] = -1;
    }
    for (int i = 0; i < patternLength; ++i) {
        lastOccurrence[static_cast<unsigned char>(pattern[i])] = i;
    }

    int skip;
    for (int i = 0; i <= textLength - patternLength; i += skip) {
        skip = 0;
        for (int j = patternLength - 1; j >= 0; --j) {
            char textChar = text[i + j];
            char patternChar = pattern[j];

            // Приводим символы к нижнему регистру перед сравнением
            if (tolower(textChar) != tolower(patternChar)) {
```

```
                    skip = max(1, j - lastOccurrence[static_cast<unsigned
char>(tolower(textChar))]);
                    break;
                }
            }
            if (skip == 0) return i; // Найдено вхождение
        }
        return -1; // Подстрока не найдена
    }
```

Описание данного метода в представленной функции boyerMooreSearch:

1. Создается таблица последних вхождений символов паттерна (lastOccurrence), где для каждого символа из алфавита (ASCII) запоминается самая правая позиция этого символа в паттерне.

2. Выполняется цикл по тексту (text) с шагом, определяемым внутри цикла, чтобы производить сдвиг паттерна на основе пропусков.

3. Во внутреннем цикле происходит сравнение символов паттерна и текста справа налево. При несовпадении символов вычисляется сдвиг (skip) на основе таблицы последних вхождений символов, чтобы исключить возможность существования вхождения в данной позиции.

4. Если сдвиг равен 0, значит вхождение подстроки найдено и функция возвращает индекс начала найденной подстроки в тексте, иначе продолжается поиск. Если вхождение не найдено, функция возвращает -1.

Таким образом, метод Боуера-Мура является эффективным алгоритмом поиска подстроки в строке за счет использования предварительной обработки и оптимизации сдвигов для исключения неудачных сравнений.

4. Описание программы

Программа для продажи авиабилетов использует открытое хеширование для хранения данных о пассажирах. Для сортировки билетов используется метод распределением. Для хранения билетов выбран циклический однонаправленный список. Для обхода дерева авиарейсов используется метод обратного обхода. Алгоритм поиска слова в тексте для авиарейсов - Боуера и Мура.

Структура программы:

1. Модуль управления продажей авиабилетов:
   - Реализация функций для добавления, удаления и поиска информации о билетах

2. Модуль хеширования данных:
   - Реализация открытого хеширования для хранения данных о пассажирах
   - Функции хеширования ключей и обработки коллизий

3. Модуль сортировки билетов:
   - Реализация метода сортировки распределением для упорядочивания данных о билетах

4. Модуль структуры данных для билетов:
   - Циклический однонаправленный список для хранения информации о билетах и пассажирах

5. Модуль обхода дерева авиарейсов:

- Реализация метода обратного обхода для управления информацией о авиарейсах

6. Модуль поиска слова в тексте для авиарейсов:

- Реализация алгоритма Боуера и Мура для эффективного поиска информации о авиарейсах

Руководство по использованию программы:

1. Запустите программу и выберите соответствующий пункт меню для выполнения нужного действия: добавление/удаление информации о пассажирах, авиарейсах и билетах.

2. Для поиска определенной информации об авиарейсе используйте функцию поиска и введите ключевое слово.

3. Для завершения операций и получения необходимой информации следуйте инструкциям программы.

Эта программа обеспечивает удобное управление продажей авиабилетов, быстрый доступ к информации о пассажирах и авиарейсах, а также эффективную обработку данных.

Листинг программы предоставлен в Приложении 1.

5. Тестирование программы

Исходные данные:

	A	B
1	31	Номер паспорта: 1234-123456, Место и дата выдачи: Piter 20.10.2004, ФИО: Pan Andrey Viktorovich, Дата рождения: 20.07.2004
2	31	Номер паспорта: 4321-654321, Место и дата выдачи: Питер 20.10.1999, ФИО: Панченко Андрей Викторович, Дата рождения: 20.07.2004
3	43	Номер паспорта: 6754-123456, Место и дата выдачи: Ospa 10.10.1999, ФИО: Ovchik Pavel Andreevich, Дата рождения: 10.10.1999

Рисунок 1 – Исходные данные хеш-таблицы пассажиров

	A	B	C	D	E	F	G	H
1	АВГ-445	Бургер	Pulkovo	Stavropol	20.10.2024	10:00:00	500	300
2	АВГ-446	Пуш	Киш	Лин	06.03.2000	9:43:24	1000	45
3	ДЛН-224	Лодка	ПМР	Россия	06.04.2022	4:00:00	100	23
4	ГОН-223	Аэрофлот	Шушкино	Черемушк	10.10.2000	20:10:55	300	100
5	АВГ-447	CheKak	Lapsha	Stalingrad	20.10.2000	10:10:10	20	1

Рисунок 2 – Исходные данные из дерева авиарейсов

	A	B	C
1	Номер Паспорта	Номер Рейса	Номер Билета
2	1112-222234	АБГ-122	123454321
3	1234-123456	АВГ-445	123456789

Рисунок 3 – Исходные данные из списков билетов

```
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 4 – Главное меню

При запуске кода мы попадает в главное меню программы, где видим множество пунктов меню. Для начала просмотрим все данные о пассажирах.

```
Введите ваш выбор: 3
31: Номер паспорта: (1234-123456) Место и дата выдачи: (Piter 20.10.2004) ФИО: (Pan Andrey Viktorovich) Дата рождения: (20.07.2004). Номер паспорта: (4321-6
54321) Место и дата выдачи: (Питер 20.10.1999) ФИО: (Панченко Андрей Викторович) Дата рождения: (20.07.2004).
43: Номер паспорта: (6754-123456) Место и дата выдачи: (Ospa 10.10.1999) ФИО: (Ovchik Pavel Andreevich) Дата рождения: (10.10.1999).
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 5 – Вывод всех данных о пассажирах

Выбрав пункт меню 3, мы увидели всю информацию о пассажирах в базе данных программы.

```
Введите ваш выбор: 1
Ввод номера паспорта (0 для выхода, формат: NNNN-NNNNNN) (N - цифра): аыф
Неверный ввод.
Введите номер паспорта заново (Формат: NNNN-NNNNNN) (N - цифра): 1234-123456
Такой номер паспорта уже существует в таблице. Введите новый номер паспорта.
Введите номер паспорта заново (Формат: NNNN-NNNNNN) (N - цифра): 3214-651234
Введите место и дату выдачи паспорта (Пример: Санкт-Петербург 20.12.2004): аыф
Неверный ввод.
Введите заново (Пример: Санкт-Петербург 20.12.2004): Тула 09.09.2004
Введите ФИО (Пример: Панченко Андрей Викторович): аыф
Неверный ввод.
Введите заново (Пример: Панченко Андрей Викторович): Панченко Виктор Викторович
Введите дату рождения (Пример: 20.07.2004): аыф
Неверный ввод.
Введите заново (Пример: 20.07.2004): 20.10.1987
Добавлено
Ввод номера паспорта (0 для выхода, формат: NNNN-NNNNNN) (N - цифра): 0
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 6 – Добавление нового пассажира в таблицу данных

Выбрав пункт меню 1, мы смогли добавить нового человека в таблицу данных программы. При добавлении данных о человеке есть множество проверок на корректность ввода. В конце, при правильном вводе всех данных о человеке можно выйти из добавления данных людей в главное меню либо продолжить добавление.

```
Введите ваш выбор: 2
Введите номер паспорта: 1111-111111
Человек с указанным номером паспорта не найден
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: 2
Введите номер паспорта: 1234-123456
Удалено
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
```

Рисунок 7 – Удаление данных о человеке из таблицы данных о пассажирах

Выбрав пункт меню 2, мы можем удалить данные о человеке из таблицы данных пассажиров. Если такого человека нету в таблице выводится соответствующее сообщение. При выборе пункта меню 4 у нас полностью очистится таблица данных о пассажирах.

```
Введите ваш выбор: 5
Введите номер паспорта: 1111-11111
Не найдено
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: 5
Введите номер паспорта: 4321-654321
Найдено в сегменте: 31
Информация о паспорте: (4321-654321) Место и дата выдачи: (Питер 20.10.1999) ФИО: (Панченко Андрей Викторович) Дата рождения: (20.07.2004).
Пассажир не имеет забронированных билетов.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 8 – Поиск данных человека по номеру паспорта в таблице данных пассажиров

Выбрав пункт меню 5, мы можем найти данные пассажира зная его номер паспорта. Если такой пассажир в таблице не найден выводится соответствующее сообщение.

```
Введите ваш выбор: 6
Введите ФИО для поиска: афы
Пассажир с указанным ФИО не найден.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: 6
Введите ФИО для поиска: Панченко Андрей Викторович
Найдено в сегменте: 31
Информация о паспорте: (4321-654321) Место и дата выдачи: (Питер 20.10.1999) ФИО: (Панченко Андрей Викторович) Дата рождения: (20.07.2004).
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 9 – Поиск данных человека по его ФИО в таблице данных пассажиров

Выбрав пункт меню 6, мы можем найти данные пассажира зная его ФИО. Если такой пассажир в таблице не найден выводится соответствующее сообщение.

```
Введите ваш выбор: 9
Список всех авиарейсов:
Номер: АВГ-445
Авиакомпания: Бургер
Аэропорт отправления: Pulkovo
Аэропорт прибытия: Stavropol
Дата отправления: 20.10.2024
Время отправления: 10:00:00
Всего мест: 500
Свободных мест: 300

Номер: АВГ-446
Авиакомпания: Пуш
Аэропорт отправления: Киш
Аэропорт прибытия: Лин
Дата отправления: 06.03.2000
Время отправления: 9:43:24
Всего мест: 1000
Свободных мест: 45

Номер: ДЛН-224
Авиакомпания: Лодка
Аэропорт отправления: ПМР
Аэропорт прибытия: Россия
Дата отправления: 06.04.2022
Время отправления: 4:00:00
Всего мест: 100
Свободных мест: 23

Номер: ГОН-223
Авиакомпания: Аэрофлот
Аэропорт отправления: Шушкино
Аэропорт прибытия: Черемушка
Дата отправления: 10.10.2000
Время отправления: 20:10:55
Всего мест: 300
Свободных мест: 100

Номер: АВГ-447
Авиакомпания: CheKak
Аэропорт отправления: Lapsha
Аэропорт прибытия: Stalingrad
Дата отправления: 20.10.2000
Время отправления: 10:10:10
Всего мест: 20
Свободных мест: 1
```

Рисунок 10 – Вывод всех авиарейсов

Выбрав пункт меню 9, мы увидели все авиарейсы, которые на данный момент есть в дерево авиарейсов.

```
Введите ваш выбор: 7
Номер авиарейса (AAA-NNN): афы
Неверный формат номера авиарейса. Повторите ввод.
Номер авиарейса (AAA-NNN): ГОН-223
Рейс с номером ГОН-223 уже существует. Повторите ввод.
Номер авиарейса (AAA-NNN): ГОЕ-111
Название авиакомпании: Летучка
Аэропорт отправления: Туда
Неверный формат аэропорта отправления. Повторите ввод.
Аэропорт отправления: Tyda
Аэропорт прибытия: Syda
Дата отправления (xx.xx.xxxx): 20
Неверный формат даты отправления. Повторите ввод.
Дата отправления (xx.xx.xxxx): 20.10.1999
Время отправления (xx:xx:xx): s
Неверный формат времени отправления. Повторите ввод.
Время отправления (xx:xx:xx): 09:09:09
Количество мест всего: f
Неверное количество мест. Повторите ввод.
Количество мест всего: 300
Количество мест свободных: f
Неверное количество свободных мест. Повторите ввод.
Количество мест свободных: 43
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 11 – Добавление нового авиарейса

Выбрав пункт меню 7, мы смогли добавить новый авиарейс со всеми соответствующими проверками.

```
Введите ваш выбор: 8
Введите номер авиарейса для удаления: ДЛН-224
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
```

Рисунок 12 – Удаление авиарейса и его данных

При выборе пункта меню 8, мы удалили авиарейс и все его данные, а также данные о билетах, которые на него были. При выборе пункта меню 10, мы очистим все данные о авиарейсах и билетов на них.

```
Введите ваш выбор: 11
Введите номер авиарейса для поиска: аыф
Авиарейс аыф не найден.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
```

```
Введите ваш выбор: 11
Введите номер авиарейса для поиска: ГОЕ-111
Информация о найденном авиарейсе:
Номер: ГОЕ-111
Авиакомпания: Летучка
Аэропорт отправления: Tyda
Аэропорт прибытия: Syda
Дата отправления: 20.10.1999
Время отправления: 09:09:09
Всего мест: 300
Свободных мест: 43
На данный авиарейс билетов не продано.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор:
```

Рисунок 13 – Поиск авиарейса по его номеру и вывод его данных

Выбрав пункт меню 11, мы смогли найти авиарейс по его номеру, если такого авиарейса нет, то выводится соответствующее сообщение.

```
Введите ваш выбор: 12
Введите фрагмент названия аэропорта прибытия: аыф
Авиарейсы по данному фрагменту не найдены.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: 12
Введите фрагмент названия аэропорта прибытия: syd
Найден авиарейс:
Номер: ГОЕ-111
Авиакомпания: Летучка
Аэропорт отправления: Tyda
Аэропорт прибытия: Syda
Дата отправления: 20.10.1999
Время отправления: 09:09:09
Всего мест: 300
Свободных мест: 43
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 14 – Поиск авиарейса по его аэропорту прибытия

Выбрав пункт меню 12, мы смогли найти авиарейс по фрагменту названия аэропорта прибытия. Если такого авиарейса не будет, то выведется сообщение.

```
Введите номер паспорта (NNNN-NNNNNN) (N - цифра): 1111-111111
Пассажир с указанным паспортом не найден в таблице.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: 13
Введите номер паспорта (NNNN-NNNNNN) (N - цифра): 4321-654321
Пассажир с указанным паспортом найден.
Введите номер авиарейса (AAA-NNN): ГОЕ-111
Введите номер билета (9 цифр): 123456789
Билет успешно выдан.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 15 – Продажа авиабилета пассажиру

При выборе пункта меню 13, мы смогли выдать пассажиру билет на авиарейс. Если такого пассажира нету для выдачи ему билета, то выводится сообщение. Все проверки на ввод имеются.

```
Введите ваш выбор: 14
Введите номер билета для удаления: 123
Билет не найден.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: 14
Введите номер билета для удаления: 123456789
Билет успешно удален.
Общее меню:
1. Регистрация нового пассажира
2. Удаление данных о пассажире
3. Просмотр всех зарегестрированных пассажиров
4. Очистка данных о пассажире
5. Поиск пассажира по номеру паспорта
6. Поиск пассажира по его ФИО
7. Добавление нового авирейса
8. Удаление сведений об авиарейсе
9. Просмотр всех авиарейсов
10.Очистка данных об авиарейсах
11.Поиск авиарейса по номеру авиарейса
12.Поиск авиарейса по фрагментам названия аэропорта прибытия
13.Регистрация продажи пассажиру авиабилета
14.Регистрация возврата пассажиру авиабилета
0. Выход
Введите ваш выбор: |
```

Рисунок 16 – Возврат билета пассажиром

Выбрав пункт меню 14, мы вернули билет пассажира удалив билет из таблицы данных. Если такого билета нет, то выводится соответствующее сообщение. При выборе пункта меню 0, мы выходим из программы.

Заключение

В ходе создания программы для управления продажей авиабилетов были применены различные методы и структуры данных, соответствующие предметной области. Открытое хеширование использовалось для эффективного хранения данных о пассажирах, что позволяет быстро находить нужную информацию. Метод сортировки распределением был выбран для упорядочивания билетов и упрощения доступа к ним.

Циклический однонаправленный список использовался в качестве структуры данных для билетов, обеспечивая удобное хранение и доступ к информации. Метод обратного обхода дерева для авиарейсов использовался для эффективного управления данными о рейсах. Алгоритм Боуера и Мура был выбран для быстрого поиска слова в тексте относительно авиарейсов.

В результате получилась программа, обеспечивающая удобное управление продажей авиабилетов, эффективное хранение и обработку информации о пассажирах и рейсах, быстрый доступ к данным и удобный поиск необходимой информации. Эти методы и структуры данных способствуют оптимизации процесса продажи авиабилетов и обеспечивают более высокую производительность программы.

Список использованных источников

1. "Язык программирования C++: специальное издание" - Брассет, Роберт. Санкт-Петербург: БХВ-Петербург, 2017.

2. "Стандарт библиотеки C++: практическое руководство" - Хорстманн, Кей С. Москва: Издательский дом Вильямс, 2019.

3. "Алгоритмы: построение и анализ" - Кормен, Томас Г., Лейзерсон, Чарльз Э., Ривест, Рональд Л., Штайн, Клиффорд. Москва: Издательский дом Вильямс, 2005.

4. "Язык программирования C++. Лекции и упражнения" - Страуструп, Бьерн. Москва: Питер, 2013.

5. "Искусство программирования. Том 3. Сортировка и поиск" - Кнут, Дональд Э. Москва: Вильямс, 2006.

6. "Алгоритмы: введение в разработку и анализ" - Кормен, Томас Г., Лейзерсон, Чарльз Э., Ривест, Рональд Л., Штайн, Клиффорд. Москва: Издательский дом Вильямс, 2019.

7. "Алгоритмы на C++" - Седжвик, Роберт, Уэйн, Кевин. Санкт-Петербург: Питер, 2004.

8. "Объектно-ориентированное программирование в C++" - Лафоре, Роберт. Москва: Издательский дом Вильямс, 2002.

9. "Программирование алгоритмов" - Боуэр, Майкл, Мур, Дж. Стенли. Москва: Издательство Техносфера, 2019.

10. SQLite3 документация — официальная документация SQLite3 для работы с базами данных, URL: https://www.sqlite.org/docs.html (дата обращения 26.01.2024).

ПРИЛОЖЕНИЕ 1

Текст программы

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <cstdlib>
#include <list>
#include <regex>

using namespace std;

//ЧАСТЬ КОДА ДЛЯ АВИАРЕЙСОВ НАЧАЛО

struct FlightData {
    string flightNumber;
    string airlineName;
    string departureAirport;
    string arrivalAirport;
    string departureDate;
    string departureTime;
    int totalSeats;
    int availableSeats;

    // Конструктор по умолчанию
    FlightData() : totalSeats(0), availableSeats(0) {}

    // Конструктор с параметрами
    FlightData(const string& number, const string& airline, const string& departure,
        const string& arrival, const string& date, const string& time,
        int total, int available)
        : flightNumber(number), airlineName(airline), departureAirport(departure),
        arrivalAirport(arrival), departureDate(date), departureTime(time),
        totalSeats(total), availableSeats(available) {}
};

struct TreeNode {
    FlightData data;
    TreeNode* left;
    TreeNode* right;

    TreeNode(const FlightData& val) : data(val), left(nullptr), right(nullptr) {}
};

class BalancedBST {
private:
    TreeNode* root;

    TreeNode* sortedArrayToBST(vector<FlightData>& sortedArray, int start, int end)
{
        if (start > end) {
            return nullptr;
        }

        int mid = (start + end) / 2;
        TreeNode* node = new TreeNode(sortedArray[mid]);

        node->left = sortedArrayToBST(sortedArray, start, mid - 1);
        node->right = sortedArrayToBST(sortedArray, mid + 1, end);

        return node;
    }

    TreeNode* rightRotate(TreeNode* y) {
```

```
        TreeNode* x = y->left;
        TreeNode* T2 = x->right;

        x->right = y;
        y->left = T2;

        return x;
    }

    TreeNode* leftRotate(TreeNode* x) {
        TreeNode* y = x->right;
        TreeNode* T2 = y->left;

        y->left = x;
        x->right = T2;

        return y;
    }

    int getBalanceFactor(TreeNode* node) {
        if (node == nullptr) {
            return 0;
        }
        return getHeight(node->left) - getHeight(node->right);
    }

    int getHeight(TreeNode* node) {
        if (node == nullptr) {
            return 0;
        }
        int leftHeight = getHeight(node->left);
        int rightHeight = getHeight(node->right);
        return 1 + max(leftHeight, rightHeight);
    }

    TreeNode* insert(TreeNode* node, const FlightData& val, bool& balanceNeeded) {
        if (node == nullptr) {
            return new TreeNode(val);
        }

        if (val.flightNumber < node->data.flightNumber) {
            node->left = insert(node->left, val, balanceNeeded);
        }
        else if (val.flightNumber > node->data.flightNumber) {
            node->right = insert(node->right, val, balanceNeeded);
        }
        else {
            return node;
        }

        int balanceFactor = getBalanceFactor(node);

        if (balanceFactor > 1 && val.flightNumber < node->left->data.flightNumber) {
            return rightRotate(node);
        }
        if (balanceFactor < -1 && val.flightNumber > node->right->data.flightNumber)
{
            return leftRotate(node);
        }
        if (balanceFactor > 1 && val.flightNumber > node->left->data.flightNumber) {
            node->left = leftRotate(node->left);
            return rightRotate(node);
        }
        if (balanceFactor < -1 && val.flightNumber < node->right->data.flightNumber)
{
```

```
            node->right = rightRotate(node->right);
            return leftRotate(node);
        }

        return node;
    }

    TreeNode* remove(TreeNode* node, const string& key, bool& balanceNeeded) {
        if (node == nullptr)
            return node;

        if (key < node->data.flightNumber)
            node->left = remove(node->left, key, balanceNeeded);
        else if (key > node->data.flightNumber)
            node->right = remove(node->right, key, balanceNeeded);
        else {
            if ((node->left == nullptr) || (node->right == nullptr)) {
                TreeNode* temp = (node->left == nullptr) ? node->right : node->left;

                if (temp == nullptr) {
                    temp = node;
                    node = nullptr;
                }
                else {
                    *node = *temp;
                }

                delete temp;
            }
            else {
                TreeNode* temp = findMin(node->right);
                node->data = temp->data;
                node->right = remove(node->right, temp->data.flightNumber,
balanceNeeded);
            }
        }

        if (node == nullptr)
            return node;

        int balance = getBalanceFactor(node);

        if (balance > 1 && getBalanceFactor(node->left) >= 0)
            return rightRotate(node);

        if (balance > 1 && getBalanceFactor(node->left) < 0) {
            node->left = leftRotate(node->left);
            return rightRotate(node);
        }

        if (balance < -1 && getBalanceFactor(node->right) <= 0)
            return leftRotate(node);

        if (balance < -1 && getBalanceFactor(node->right) > 0) {
            node->right = rightRotate(node->right);
            return leftRotate(node);
        }

        return node;
    }

    TreeNode* findMin(TreeNode* node) {
        while (node->left != nullptr)
            node = node->left;
        return node;
```

```
    }

    bool isEqualFlightNumber(const string& str1, const string& str2) {
        return str1 == str2;
    }

    bool search(TreeNode* node, const string& val, int& steps) {
        if (node == nullptr)
            return false;

        steps++;

        if (node->data.flightNumber == val)
            return true;
        else if (val < node->data.flightNumber)
            return search(node->left, val, steps);
        else
            return search(node->right, val, steps);
    }

    void printTree(TreeNode* node, int indent = 0) {
        if (node == nullptr)
            return;
        printTree(node->right, indent + 4);
        for (int i = 0; i < indent; i++)
            cout << " ";
        cout << node->data.flightNumber << "\n";
        printTree(node->left, indent + 4);
    }


    void preorderTraversal(TreeNode* node, vector<FlightData>& result) {
        if (node == nullptr)
            return;
        result.push_back(node->data);
        preorderTraversal(node->left, result);
        preorderTraversal(node->right, result);
    }

    void reverseInorderTraversal(TreeNode* node, vector<FlightData>& leftPart,
vector<FlightData>& rightPart) {
        if (node == nullptr)
            return;

        // Обход левой части: начинайте с левого поддерева
        reverseInorderTraversal(node->left, leftPart, rightPart);
        // Если текущий узел меньше корня, добавляем в левую часть
        if (node->data.flightNumber < root->data.flightNumber) {
            leftPart.push_back(node->data);  // Сохраняем значение узла
        }
        // Если текущий узел больше корня, добавляем в правую часть
        else if (node->data.flightNumber > root->data.flightNumber) {
            rightPart.push_back(node->data);  // Сохраняем значение узла
        }

        // Обход правой части: начинайте с правого поддерева
        reverseInorderTraversal(node->right, leftPart, rightPart);
    }
    TreeNode* balanceTree(vector<FlightData>& sortedArray, int start, int end) {
        if (start > end) {
            return nullptr;
        }

        int mid = (start + end) / 2;
        TreeNode* node = new TreeNode(sortedArray[mid]);
```

```
        node->left = balanceTree(sortedArray, start, mid - 1);
        node->right = balanceTree(sortedArray, mid + 1, end);

        return node;
    }

    void clear(TreeNode* node) {
        if (node == nullptr)
            return;

        clear(node->left);
        clear(node->right);
        delete node;
    }

public:
    BalancedBST() : root(nullptr) {}

    void insert(const FlightData& val) {
        vector<FlightData> sortedElements;
        preorderTraversal(root, sortedElements);
        sortedElements.push_back(val);
        sort(sortedElements.begin(), sortedElements.end(),
            [](const FlightData& a, const FlightData& b) {
                return a.flightNumber < b.flightNumber;
            });
        root = sortedArrayToBST(sortedElements, 0, sortedElements.size() - 1);
    }

    void remove(const string& val) {
        bool balanceNeeded = false;
        root = remove(root, val, balanceNeeded);
        if (balanceNeeded) {
            vector<FlightData> sortedElements;
            preorderTraversal(root, sortedElements);
            sort(sortedElements.begin(), sortedElements.end(),
                [](const FlightData& a, const FlightData& b) {
                    return a.flightNumber < b.flightNumber;
                });
            root = balanceTree(sortedElements, 0, sortedElements.size() - 1);
        }
    }

    bool search(const string& val, int& steps) {
        steps = 0;
        return search(root, val, steps);
    }

    void printTree() {
        printTree(root);
    }

    vector<FlightData> reverseInorderTraversal() {
        vector<FlightData> leftPart;
        vector<FlightData> rightPart;
        reverseInorderTraversal(root, leftPart, rightPart);

        // Соединяем левую и правую части, добавляем корень в конце
        leftPart.insert(leftPart.end(), rightPart.rbegin(), rightPart.rend());
        leftPart.push_back(root->data);
        return leftPart;
    }

    FlightData find(const string& val) {
```

```
        FlightData result;
        find(root, val, result);
        return result;
    }

    void find(TreeNode* node, const string& val, FlightData& result) {
        if (node == nullptr)
            return;

        if (node->data.flightNumber == val) {
            result = node->data;
            return;
        }
        else if (val < node->data.flightNumber)
            find(node->left, val, result);
        else
            find(node->right, val, result);
    }

    void loadFromCSV(const string& filename) {
        ifstream file(filename);
        if (!file.is_open()) {
            cerr << "Ошибка открытия файла: " << filename << endl;
            return;
        }

        string line;
        while (getline(file, line)) {
            stringstream ss(line);
            FlightData flight;

            // Извлекаем все поля из строки, разделенные символом ';'
            getline(ss, flight.flightNumber, ';');
            getline(ss, flight.airlineName, ';');
            getline(ss, flight.departureAirport, ';');
            getline(ss, flight.arrivalAirport, ';');
            getline(ss, flight.departureDate, ';');
            getline(ss, flight.departureTime, ';');
            ss >> flight.totalSeats;
            ss.ignore();  // Пропускаем символ разделителя ';'
            ss >> flight.availableSeats;

            insert(flight);
        }

        file.close();
    }

    // Функция сохранения в csv файл
    void saveToCSV(const string& filename) {
        ofstream file(filename);
        if (!file.is_open()) {
            cerr << "Error opening file: " << filename << endl;
            return;
        }

        vector<FlightData> allFlights = reverseInorderTraversal();
        for (const FlightData& flight : allFlights) {
            file << flight.flightNumber << ';' << flight.airlineName << ';' <<
flight.departureAirport
                << ';' << flight.arrivalAirport << ';' << flight.departureDate <<
';' << flight.departureTime
                << ';' << flight.totalSeats << ';' << flight.availableSeats << '\n';
        }
```

```
        file.close();
    }

    void clear() {
        clear(root);
        root = nullptr;
    }
};

int boyerMooreSearch(const string& text, const string& pattern) {
    int textLength = text.length();
    int patternLength = pattern.length();

    if (patternLength == 0) return 0;

    const int CHAR_SET_SIZE = 256; // Размер алфавита ASCII
    int lastOccurrence[CHAR_SET_SIZE]; // Таблица последних вхождений

    // Заполняем таблицу последних вхождений
    for (int i = 0; i < CHAR_SET_SIZE; ++i) {
        lastOccurrence[i] = -1;
    }
    for (int i = 0; i < patternLength; ++i) {
        lastOccurrence[static_cast<unsigned char>(pattern[i])] = i;
    }

    int skip;
    for (int i = 0; i <= textLength - patternLength; i += skip) {
        skip = 0;
        for (int j = patternLength - 1; j >= 0; --j) {
            char textChar = text[i + j];
            char patternChar = pattern[j];

            // Приводим символы к нижнему регистру перед сравнением
            if (tolower(textChar) != tolower(patternChar)) {
                skip = max(1, j - lastOccurrence[static_cast<unsigned
char>(tolower(textChar))]);
                break;
            }
        }
        if (skip == 0) return i; // Найдено вхождение
    }
    return -1; // Подстрока не найдена
}

// Функция для проверки формата ключа (номера авиарейса)
bool validateFlightKey(const string& key) {
    regex pattern("^[А-Я]{3}-[0-9]{3}$");
    return regex_match(key, pattern);
}

// Функция для проверки формата времени
bool validateTime(const string& time) {
    regex pattern("^([0-1][0-9]|2[0-3]):[0-5][0-9]:[0-5][0-9]$");
    return regex_match(time, pattern);
}

// Функция для проверки формата даты
bool validateDate(const string& date) {
    regex pattern("^\\d{2}\\.\\d{2}\\.\\d{4}$");
    return regex_match(date, pattern);
}

//ЧАСТЬ КОДА ДЛЯ АВИАРЕЙСОВ КОНЕЦ
```

```
//ЧАСТЬ КОДА ДЛЯ БИЛЕТОВ НАЧАЛО

struct Ticket {
    string passportNumber;
    string flightNumber;
    string ticketNumber;
};

// Функция для сортировки списка методом распределения
void bucketSort(vector<Ticket>& tickets) {
    sort(tickets.begin(), tickets.end(),
        [](const Ticket& a, const Ticket& b) {
            return a.ticketNumber < b.ticketNumber;
        });
}

// Функция для сохранения списка в CSV файл
void saveToCSV(const vector<Ticket>& tickets, const string& filename) {
    ofstream file(filename);

    if (file.is_open()) {
        file << "Номер Паспорта;Номер Рейса;Номер Билета\n";

        for (const Ticket& ticket : tickets) {
            file << ticket.passportNumber << ";" << ticket.flightNumber << ";" <<
ticket.ticketNumber << "\n";
        }

        file.close();
    }
    else {
        cerr << "Невозможно открыть файл: " << filename << endl;
    }
}

// Функция для открытия CSV файла и чтения данных
void readCSV(const string& filename, vector<Ticket>& tickets) {
    ifstream file(filename);

    if (file.is_open()) {
        string line;
        getline(file, line);

        while (getline(file, line)) {
            stringstream ss(line);
            string passport, flight, ticket;

            getline(ss, passport, ';');
            getline(ss, flight, ';');
            getline(ss, ticket, ';');

            tickets.push_back({ passport, flight, ticket });
        }

        file.close();
    }
    else {
        cerr << "Невозможно открыть файл: " << filename << endl;
    }
}

bool validatePassportNumber(const string& passportNumber) {
    regex pattern("^[0-9]{4}-[0-9]{6}$");
    return regex_match(passportNumber, pattern);
}
```

```
bool validateTicketNumber(const string& ticketNumber) {
    regex pattern("^[0-9]{9}$");
    return regex_match(ticketNumber, pattern);
}

bool isTicketExists(const vector<Ticket>& entries, const string& ticketNumber) {
    for (const auto& entry : entries) {
        if (entry.ticketNumber == ticketNumber) {
            return true;
        }
    }
    return false;
}

//ЧАСТЬ КОДА ДЛЯ БИЛЕТОВ КОНЕЦ

//ЧАСТЬ КОДА ДЛЯ ПАССАЖИРОВ НАЧАЛО

const int n = 3000;
const int m1 = 12;

struct KeyValuePair {
    char key[m1];
    char placeAndDate[100];
    char fullName[100];
    char birthDate[20];
    bool isDeleted;
};

int Hash(const char key[m1]) {
    int hash = 0;

    for (int i = 0; i < m1; i++) {
        if (isdigit(key[i])) {
            hash += key[i] - '0'; // Преобразуем символ-цифру в целое число
        }
    }

    hash %= n;
    return hash;
}

void save(list<KeyValuePair> t[]) {
    fstream file("hash.csv", ios_base::out);
    if (!file.is_open()) {
        cerr << "Не удалось открыть файл\n";
        return;
    }

    for (int i = 0; i < n; i++) {
        for (const auto& entry : t[i]) {
            if (!entry.isDeleted) {
                file << i << ";";
                file << "Номер паспорта: " << entry.key << ", " << "Место и дата
выдачи: "
                    << entry.placeAndDate << ", " << "ФИО: " << entry.fullName
                    << ", " << "Дата рождения: " << entry.birthDate << '\n';
            }
        }
    }

    file.close();
}
```

```
void load(list<KeyValuePair> t[]) {
    ifstream file("hash.csv");
    if (!file.is_open()) {
        cerr << "Не удалось открыть файл\n";
        return;
    }

    string line;
    while (getline(file, line)) {
        if (line.empty()) {
            continue;
        }

        stringstream ss(line);

        int id;
        ss >> id;
        ss.ignore(); // Пропускаем точку с запятой

        KeyValuePair newEntry;
        string key;

        while (ss >> key) {
            if (key == ";") {
                break; // Завершаем чтение при встрече точки с запятой
            }

            if (key == "Номер") {
                ss >> key; // Пропускаем "паспорта:"
                ss.ignore();
                getline(ss, key, ','); // Читаем номер паспорта до запятой
                strcpy_s(newEntry.key, m1, key.c_str());
            }
            else if (key == "Место") {
                ss >> key; // Пропускаем "и"
                ss >> key; // Читаем "дата"
                ss >> key; // Читаем "выдачи"
                ss.ignore();
                getline(ss, key, ','); // Читаем место выдачи паспорта до запятой
                strcpy_s(newEntry.placeAndDate, 100, key.c_str());
            }
            else if (key == "ФИО:") {
                ss.ignore();
                getline(ss, key, ','); // Читаем ФИО до запятой
                strcpy_s(newEntry.fullName, 100, key.c_str());
            }
            else if (key == "Дата") {
                ss >> key; // Пропускаем "рождения:"
                ss.ignore();
                getline(ss, key, ','); // Читаем дату рождения до запятой
                strcpy_s(newEntry.birthDate, 20, key.c_str());
            }
        }

        newEntry.isDeleted = false;
        t[id].push_back(newEntry);
    }

    file.close();
}

bool validatePassportKey(const string& key) {
    regex pattern("^[0-9]{4}-[0-9]{6}$");
    return regex_match(key, pattern);
}
```

```
bool validateFullName(const string& fullName) {
    regex pattern("^[А-Яа-яА-Za-z]+ [А-Яа-яА-Za-z]+ [А-Яа-яА-Za-z]+$");
    return regex_match(fullName, pattern);
}

bool validatePlaceAndDate(const string& placeAndDate) {
    regex pattern("^[А-Яа-яА-Za-z\\-]+ [0-9]{2}\\.[0-9]{2}\\.[0-9]{4}$");
    return regex_match(placeAndDate, pattern);
}

bool validateBirthDate(const string& birthDate) {
    regex pattern("^[0-9]{2}.[0-9]{2}.[0-9]{4}$");
    return regex_match(birthDate, pattern);
}

bool isKeyExists(const list<KeyValuePair>& entries, const char* key) {
    for (const auto& entry : entries) {
        if (!strcmp(entry.key, key) && !entry.isDeleted) {
            return true;
        }
    }
    return false;
}

bool add(list<KeyValuePair> t[], const string& k, const string& placeAndDate, const
string& fullName, const string& birthDate) {
    if (!validatePassportKey(k)) {
        cout << "Неверный формат ключа. Используйте формат NNNN-NNNNNN.\n";
        return false;
    }

    char key[m1];
    strcpy_s(key, m1, k.c_str());
    int id = Hash(key);

    KeyValuePair newEntry;
    strcpy_s(newEntry.key, m1, key);
    strcpy_s(newEntry.placeAndDate, 100, placeAndDate.c_str());
    strcpy_s(newEntry.fullName, 100, fullName.c_str());
    strcpy_s(newEntry.birthDate, 20, birthDate.c_str());
    newEntry.isDeleted = false;

    t[id].push_back(newEntry);
    return true;
}

KeyValuePair* find(list<KeyValuePair> t[], char k[m1]) {
    int id = Hash(k);

    for (int i = 0; i < n; i++) {
        for (auto& entry : t[i]) {
            if (!strcmp(entry.key, k) && !entry.isDeleted) {
                return &entry;
            }
        }
    }

    return nullptr;
}

bool del(list<KeyValuePair> t[], const string& k) {
    int id = Hash(k.c_str());

    for (auto& entry : t[id]) {
```

```
        if (!strcmp(entry.key, k.c_str())) {
            entry.isDeleted = true;
            return true;
        }
    }

    return false;
}

void print(list<KeyValuePair> t[]) {
    for (int i = 0; i < n; i++) {
        if (!t[i].empty()) {
            cout << i << ": ";
            for (const auto& entry : t[i]) {
                if (!entry.isDeleted) {
                    cout << "Номер паспорта: (" << entry.key << ") Место и дата
выдачи: (" << entry.placeAndDate << ") ФИО: (" << entry.fullName << ") Дата
рождения: (" << entry.birthDate << "). ";
                }
            }
            cout << endl;
        }
    }
}

//ЧАСТЬ КОДА ДЛЯ ПАССАЖИРОВ КОНЕЦ

int main() {
    system("chcp 1251 > nul");
    srand(time(NULL));
    list<KeyValuePair> table[n];
    char key[m1];
    string placeAndDate;
    string fullName;
    string birthDate;

    load(table);
    char x[100];
    BalancedBST tree;

    tree.loadFromCSV("flight.csv");

    int choice;
    FlightData value;
    int steps;
    vector<Ticket> ticketList;
    readCSV("ticket.csv", ticketList);

    while (true) {
        cout << "Общее меню:\n";
        cout << "1. Регистрация нового пассажира\n";
        cout << "2. Удаление данных о пассажире\n";
        cout << "3. Просмотр всех зарегестрированных пассажиров\n";
        cout << "4. Очистка данных о пассажире\n";
        cout << "5. Поиск пассажира по номеру паспорта\n";
        cout << "6. Поиск пассажира по его ФИО\n";
        cout << "7. Добавление нового авирейса\n";
        cout << "8. Удаление сведений об авиарейсе\n";
        cout << "9. Просмотр всех авиарейсов\n";
        cout << "10.Очистка данных об авиарейсах\n";
        cout << "11.Поиск авиарейса по номеру авиарейса\n";
        cout << "12.Поиск авиарейса по фрагментам названия аэропорта прибытия\n";
        cout << "13.Регистрация продажи пассажиру авиабилета\n";
        cout << "14.Регистрация возврата пассажиру авиабилета\n";
        cout << "0. Выход\n";
```

```
        cout << "Введите ваш выбор: ";
        cin >> choice;

        switch (choice) {
        case 1: {
            bool t;
            int q = 1;
            int r = 0;
            while (q != 0) {
                for (int i = 0; i < n; i++) {
                    if (!table[i].empty()) {
                        q = 0;
                    }
                    else {
                        q = 1;
                    }
                }
                if (q == 0) {
                    cout << "Переполнение!\n";
                    break;
                }
                else {
                    // Ввод ключа
                    cout << "Ввод номера паспорта (0 для выхода, формат: NNNN-
NNNNNN) (N - цифра): ";
                    cin >> key;
                    if (key[0] == '0') {
                        break; // Выходим из цикла добавления элементов в хеш-
таблицу
                    }

                    while (!validatePassportKey(key) ||
isKeyExists(table[Hash(key)], key)) {
                        if (!validatePassportKey(key)) {
                            cout << "Неверный ввод.\n";
                        }
                        else {
                            cout << "Такой номер паспорта уже существует в таблице.
Введите новый номер паспорта.\n";
                        }
                        cout << "Введите номер паспорта заново (Формат: NNNN-NNNNNN)
(N - цифра): ";
                        cin.ignore();  // Очищаем буфер после ввода ключа
                        cin >> key;
                    }

                    // Ввод места и даты выдачи паспорта
                    cout << "Введите место и дату выдачи паспорта (Пример: Санкт-
Петербург 20.12.2004): ";
                    cin.ignore(); // Очищаем буфер перед использованием getline
                    getline(cin, placeAndDate);

                    while (!validatePlaceAndDate(placeAndDate)) {
                        cout << "Неверный ввод.\n";
                        cout << "Введите заново (Пример: Санкт-Петербург
20.12.2004): ";
                        getline(cin, placeAndDate);
                    }

                    cout << "Введите ФИО (Пример: Панченко Андрей Викторович): ";
                    getline(cin, fullName);

                    // Ввод ФИО
                    while (!validateFullName(fullName)) {
                        cout << "Неверный ввод.\n";
```

```
                        cout << "Введите заново (Пример: Панченко Андрей
Викторович): ";
                        getline(cin, fullName);
                    }

                    cout << "Введите дату рождения (Пример: 20.07.2004): ";
                    getline(cin, birthDate);

                    // Ввод даты рождения
                    while (!validateBirthDate(birthDate)) {
                        cout << "Неверный ввод.\n";
                        cout << "Введите заново (Пример: 20.07.2004): ";
                        getline(cin, birthDate);
                    }

                    // Автоматически добавляем ". " в конце даты рождения
                    //birthDate += ". ";

                    if (add(table, key, placeAndDate, fullName, birthDate)) {
                        cout << "Добавлено\n";
                        save(table);
                    }
                    else {
                        cout << "Ошибка\n";
                    }
                }
            }
            break;
        }
        case 2: {
            cout << "Введите номер паспорта: ";
            cin >> key;

            // Найти человека в хеш-таблице
            KeyValuePair* person = find(table, key);

            if (person) {
                // Найти и удалить связанные билеты
                vector<Ticket>::iterator it = remove_if(ticketList.begin(),
ticketList.end(), [key](const Ticket& ticket) {
                    return ticket.passportNumber == key;
                    });

                // Удалить билеты из вектора
                ticketList.erase(it, ticketList.end());

                // Удалить данные о человеке из хеш-таблицы
                if (del(table, key)) {
                    cout << "Удалено\n";
                }
                else {
                    cout << "Ошибка при удалении данных человека\n";
                }
            }
            else {
                cout << "Человек с указанным номером паспорта не найден\n";
            }
            break;
        }
        case 3: {
            print(table);
            break;
        }
        case 4: {
            for (int i = 0; i < n; i++) {
```

```
                table[i].clear(); // Очищаем каждый список в таблице
            }
            cout << "Все данные о пассажирах очищены.\n";
            save(table);
            break;
        }

        case 5: {
            cout << "Введите номер паспорта: ";
            cin >> key;
            KeyValuePair* result = find(table, key);

            if (result) {
                cout << "Найдено в сегменте: " << Hash(key) << "\n";
                cout << "Информация о паспорте: (" << result->key << ") " << "Место
и дата выдачи: (" << result->placeAndDate << ") ФИО: (" << result->fullName << ")
Дата рождения: (" << result->birthDate << ")." << "\n";

                // Поиск билетов для данного пассажира
                vector<string> bookedFlights;
                for (const Ticket& ticket : ticketList) {
                    if (ticket.passportNumber == key) {
                        bookedFlights.push_back(ticket.flightNumber);
                    }
                }

                if (!bookedFlights.empty()) {
                    cout << "Билеты на авиарейсы:\n";
                    for (const string& flightNumber : bookedFlights) {
                        cout << flightNumber << "\n";
                    }
                }
                else {
                    cout << "Пассажир не имеет забронированных билетов.\n";
                }
            }
            else {
                cout << "Не найдено\n";
            }
            break;
        }
        case 6: {
            string searchFullName;
            cout << "Введите ФИО для поиска: ";
            cin.ignore();
            getline(cin, searchFullName);

            bool found = false;

            for (int i = 0; i < n; i++) {
                for (const auto& entry : table[i]) {
                    if (!entry.isDeleted && entry.fullName == searchFullName) {
                        found = true;
                        cout << "Найдено в сегменте: " << i << "\n";
                        cout << "Информация о паспорте: (" << entry.key << ") Место
и дата выдачи: (" << entry.placeAndDate << ") ФИО: (" << entry.fullName << ") Дата
рождения: (" << entry.birthDate << ")." << "\n";
                    }
                }
            }

            if (!found) {
                cout << "Пассажир с указанным ФИО не найден.\n";
            }
```

```
            break;
        }
        case 7:
            do {
                cout << "Номер авиарейса (AAA-NNN): ";
                cin >> value.flightNumber;

                // Проверяем, существует ли уже рейс с таким номером
                if (tree.search(value.flightNumber, steps)) {
                    cout << "Рейс с номером " << value.flightNumber << " уже
существует. Повторите ввод." << endl;
                }
                // Проверяем формат номера авиарейса
                else if (!validateFlightKey(value.flightNumber)) {
                    cout << "Неверный формат номера авиарейса. Повторите ввод." <<
endl;
                }
            } while (!validateFlightKey(value.flightNumber) ||
tree.search(value.flightNumber, steps));

            cout << "Название авиакомпании: ";
            cin >> value.airlineName;
            do {
                cout << "Аэропорт отправления: ";
                cin >> value.departureAirport;

                // Проверка формата аэропорта отправления
                if (!regex_match(value.departureAirport, regex("^[A-Za-z]+$"))) {
                    cout << "Неверный формат аэропорта отправления. Повторите ввод."
<< endl;
                }
            } while (!regex_match(value.departureAirport, regex("^[A-Za-z]+$")));

            do {
                cout << "Аэропорт прибытия: ";
                cin >> value.arrivalAirport;

                // Проверка формата аэропорта прибытия
                if (!regex_match(value.arrivalAirport, regex("^[A-Za-z]+$"))) {
                    cout << "Неверный формат аэропорта прибытия. Повторите ввод." <<
endl;
                }
            } while (!regex_match(value.arrivalAirport, regex("^[A-Za-z]+$")));

            do {
                cout << "Дата отправления (xx.xx.xxxx): ";
                cin >> value.departureDate;
                // Добавляем проверку для даты отправления
                if (!validateDate(value.departureDate)) {
                    cout << "Неверный формат даты отправления. Повторите ввод." <<
endl;
                }
            } while (!validateDate(value.departureDate));

            do {
                cout << "Время отправления (xx:xx:xx): ";
                cin >> value.departureTime;
                // Добавляем проверку для времени отправления
                if (!validateTime(value.departureTime)) {
                    cout << "Неверный формат времени отправления. Повторите ввод."
<< endl;
                }
            } while (!validateTime(value.departureTime));

            do {
```

```
                cout << "Количество мест всего: ";
                cin >> value.totalSeats;

                if (cin.fail() || value.totalSeats <= 0) {
                    cout << "Неверное количество мест. Повторите ввод." << endl;
                    cin.clear();  // Очистка флагов ошибок
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');  // Очистка
буфера ввода
                }
                else {
                    break;  // Выход из цикла при успешном вводе
                }
            } while (true);

            do {
                cout << "Количество мест свободных: ";
                cin >> value.availableSeats;

                if (cin.fail() || value.availableSeats < 0 || value.availableSeats >
value.totalSeats) {
                    cout << "Неверное количество свободных мест. Повторите ввод." <<
endl;
                    cin.clear();  // Очистка флагов ошибок
                    cin.ignore(numeric_limits<streamsize>::max(), '\n');  // Очистка
буфера ввода
                }
                else {
                    break;  // Выход из цикла при успешном вводе
                }
            } while (true);
            tree.insert(value);
            break;
        case 8: {
            FlightData value;
            cout << "Введите номер авиарейса для удаления: ";
            cin >> value.flightNumber;

            // Удаляем все билеты, связанные с удаляемым авиарейсом
            vector<Ticket>::iterator it = remove_if(ticketList.begin(),
ticketList.end(), [value](const Ticket& ticket) {
                return ticket.flightNumber == value.flightNumber;
                });

            // Удаление билетов из вектора
            ticketList.erase(it, ticketList.end());

            // Удаление авиарейса из дерева
            tree.remove(value.flightNumber);
            break;
        }
        case 9: {
            vector<FlightData> allFlights = tree.reverseInorderTraversal();
            cout << "Список всех авиарейсов:\n";
            for (const FlightData& flight : allFlights) {
                cout << "Номер: " << flight.flightNumber << "\n"
                    << "Авиакомпания: " << flight.airlineName << "\n"
                    << "Аэропорт отправления: " << flight.departureAirport << "\n"
                    << "Аэропорт прибытия: " << flight.arrivalAirport << "\n"
                    << "Дата отправления: " << flight.departureDate << "\n"
                    << "Время отправления: " << flight.departureTime << "\n"
                    << "Всего мест: " << flight.totalSeats << "\n"
                    << "Свободных мест: " << flight.availableSeats << "\n\n";
            }
            break;
        }
```

```
        case 10:
            tree.clear();
            cout << "Все авиарейсы и их данные удалены.\n";
            tree.saveToCSV("flight.csv");
            break;
        case 11: {
            cout << "Введите номер авиарейса для поиска: ";
            cin >> value.flightNumber;
            FlightData flightInfo = tree.find(value.flightNumber);

            if (!flightInfo.flightNumber.empty()) {
                cout << "Информация о найденном авиарейсе:\n"
                    << "Номер: " << flightInfo.flightNumber << "\n"
                    << "Авиакомпания: " << flightInfo.airlineName << "\n"
                    << "Аэропорт отправления: " << flightInfo.departureAirport <<
"\n"
                    << "Аэропорт прибытия: " << flightInfo.arrivalAirport << "\n"
                    << "Дата отправления: " << flightInfo.departureDate << "\n"
                    << "Время отправления: " << flightInfo.departureTime << "\n"
                    << "Всего мест: " << flightInfo.totalSeats << "\n"
                    << "Свободных мест: " << flightInfo.availableSeats << endl;

                // Поиск пассажиров, купивших билеты на этот авиарейс
                vector<string> passengers;
                for (const Ticket& ticket : ticketList) {
                    if (ticket.flightNumber == value.flightNumber) {
                        passengers.push_back(ticket.passportNumber);
                    }
                }

                if (!passengers.empty()) {
                    cout << "Пассажиры, купившие билеты на этот авиарейс:\n";
                    for (const string& passportNumber : passengers) {
                        cout << "Номер паспорта: " << passportNumber << "\n";
                    }
                }
                else {
                    cout << "На данный авиарейс билетов не продано.\n";
                }
            }
            else {
                cout << "Авиарейс " << value.flightNumber << " не найден." << endl;
            }
            break;
        }
        case 12: {
            cout << "Введите фрагмент названия аэропорта прибытия: ";
            string fragment;
            cin >> fragment;

            // Поиск по фрагменту с использованием алгоритма Бойера-Мура
            vector<FlightData> allFlights = tree.reverseInorderTraversal();
            bool found = false;
            for (const FlightData& flight : allFlights) {
                if (boyerMooreSearch(flight.arrivalAirport, fragment) != -1) {
                    cout << "Найден авиарейс:\n"
                        << "Номер: " << flight.flightNumber << "\n"
                        << "Авиакомпания: " << flight.airlineName << "\n"
                        << "Аэропорт отправления: " << flight.departureAirport <<
"\n"
                        << "Аэропорт прибытия: " << flight.arrivalAirport << "\n"
                        << "Дата отправления: " << flight.departureDate << "\n"
                        << "Время отправления: " << flight.departureTime << "\n"
                        << "Всего мест: " << flight.totalSeats << "\n"
                        << "Свободных мест: " << flight.availableSeats << endl;
```

```
                found = true;
            }
        }

        if (!found) {
            cout << "Авиарейсы по данному фрагменту не найдены." << endl;
        }
        break;
    }
    case 13: {
        Ticket newTicket;

        cout << "Введите номер паспорта (NNNN-NNNNNN) (N - цифра): ";
        cin >> key;

        while (!validatePassportNumber(key)) {
            cout << "Неверный ввод номера паспорта.\n";
            cout << "Введите номер паспорта заново (Формат: NNNN-NNNNNN) (N -
цифра): ";
            cin >> key;
        }

        // Проверка наличия паспорта в таблице пассажиров
        KeyValuePair* passenger = find(table, key);

        if (passenger) {
            newTicket.passportNumber = key;
            cout << "Пассажир с указанным паспортом найден.\n";

            FlightData flight;  // Переменная flight объявлена здесь

            do {
                cout << "Введите номер авиарейса (AAA-NNN): ";
                cin >> value.flightNumber;

                // Поиск информации о рейсе в дереве
                flight = tree.find(value.flightNumber);  // Используем ту же
переменную flight, объявленную выше

                // Проверка наличия рейса и свободных мест
                if (flight.flightNumber != "") {
                    if (flight.availableSeats > 0) {
                        newTicket.flightNumber = value.flightNumber;
                        flight.availableSeats--; // Уменьшаем количество
свободных мест на рейсе

                        cout << "Введите номер билета (9 цифр): ";
                        cin >> newTicket.ticketNumber;

                        while (!validateTicketNumber(newTicket.ticketNumber) ||
isTicketExists(ticketList, newTicket.ticketNumber)) {
                            if (!validateTicketNumber(newTicket.ticketNumber)) {
                                cout << "Неверный ввод номера билета.\n";
                            }
                            else {
                                cout << "Такой номер билета уже существует в
таблице. Введите новый номер билета.\n";
                            }
                            cout << "Введите номер билета заново (Формат: 9
цифр): ";
                            cin >> newTicket.ticketNumber;
                        }

                        ticketList.push_back(newTicket);
```

```
                            // Упорядочиваем список билетов по номеру авиабилета
                            bucketSort(ticketList);

                            cout << "Билет успешно выдан.\n";
                        }
                        else {
                            cout << "Извините, свободных мест на данном рейсе
нет.\n";
                        }
                    }
                    else {
                        cout << "Рейс с указанным номером не найден.\n";
                    }

                } while (!validateFlightKey(value.flightNumber) ||
flight.flightNumber == "");
            }
            else {
                cout << "Пассажир с указанным паспортом не найден в таблице.\n";
            }

            break;
        }
        case 14:
        {
            string ticketNumberInput;
            cout << "Введите номер билета для удаления: ";
            cin >> ticketNumberInput;

            auto it = find_if(ticketList.begin(), ticketList.end(),
                [ticketNumberInput](const Ticket& ticket) {
                    return ticket.ticketNumber == ticketNumberInput;
                });

            if (it != ticketList.end()) {
                ticketList.erase(it);
                cout << "Билет успешно удален.\n";
                // Упорядочиваем список билетов по номеру авиабилета после удаления
                bucketSort(ticketList);
            }
            else {
                cout << "Билет не найден.\n";
            }
            break;
        }
        case 0:
            save(table);
            tree.saveToCSV("flight.csv");
            saveToCSV(ticketList, "ticket.csv");
            return 0; // Выход из программы
        default:
            cout << "Некорректный выбор. Пожалуйста, введите правильную опцию.\n";
        }
    }
}
```